Chapter 8 worked solutions

From the Original Block to Modern Decoders

This guide covers Exercises 1–5 and 12. Exercises 6–11 are in the companion Jupyter notebook, which is also available as a PDF. The first five exercises follow one calculation at a time through the residual path, the normalizer, the feed-forward layer, and the query–key dot product.

1. Move the normalizer

Start with the token vector $(3, 4)$. A **residual path** carries a copy of that vector to an addition. The sublayer supplies the other input to the addition, called the **update**. Here the sublayer returns $(0, 0)$ for every input. To find the block output, follow each copy of the input and locate the normalization step.

We will need the same LayerNorm calculation in both arrangements. LayerNorm takes the mean of the features in this token and subtracts it from each coordinate. It divides the centered coordinates by the square root of their mean square, with the given $\epsilon = 0$. For $(3, 4)$ the intermediate quantities are

Quantity	Calculation	Result
Mean	$(3 + 4)/2$	3.5
Centered coordinates	$(3 - 3.5,\ 4 - 3.5)$	$(-0.5,\ 0.5)$
Variance	$((-0.5)^2 + (0.5)^2)/2$	0.25
Denominator	$\sqrt{0.25 + 0}$	0.5

There are two features, so LayerNorm divides by two in the variance calculation. The final gain multiplies each divided coordinate by one, and the offset adds zero. The normalized vector is therefore $(-0.5, 0.5)/0.5 = (-1, 1)$.

In **post-norm**, the sublayer receives $(3, 4)$ and returns $(0, 0)$. The residual branch also carries $(3, 4)$. The addition returns $(3, 4) + (0, 0) = (3, 4)$, and then LayerNorm operates on that sum. The block returns $(-1, 1)$.

In **pre-norm**, one branch applies LayerNorm to $(3, 4)$ and sends $(-1, 1)$ to the sublayer. The sublayer still returns $(0, 0)$. The residual branch carries the original $(3, 4)$ all the way to the addition. The block returns $(3, 4)$. The same LayerNorm result appears in both arrangements. In post-norm it is the block output, and in pre-norm it is the input to the update branch.

2. Rescale the same token

The two inputs $(3, 4)$ and $(6, 8)$ have the same relative proportions. The second is twice the first. RMSNorm measures the size of each input by squaring its coordinates, taking their mean, and taking the square root. It divides both coordinates by this one number. After division, each coordinate receives its own learned multiplier, called a **gain**. In this exercise the first coordinate is doubled and the second is halved.

For $(3, 4)$ the mean square is $(9 + 16)/2 = 12.5$. With $\epsilon = 0$, division gives $(3, 4)/\sqrt{12.5}$. Applying the gains gives

$$\left(2\frac{3}{\sqrt{12.5}},\ \frac{1}{2}\frac{4}{\sqrt{12.5}}\right) = \frac{(6,2)}{\sqrt{12.5}}.$$

For $(6, 8)$ the mean square is $(36 + 64)/2 = 50$. Applying the same rule gives

$$\left(2\frac{6}{\sqrt{50}},\ \frac{1}{2}\frac{8}{\sqrt{50}}\right) = \frac{(12,4)}{\sqrt{50}} = \frac{(6,2)}{\sqrt{12.5}}.$$

The equality holds because doubling the coordinates multiplies their squares by four. The square root then doubles the denominator, so the factor of two cancels. More generally, for any nonzero input and positive scaling factor c, the same cancellation follows from $\sqrt{c^2 s} = c\sqrt{s}$, where s is the original mean square. The gains are the same in both calculations and leave that equality intact. Both outputs are approximately $(1.697056, 0.565685)$.

We can check which stage has mean square one. Before the gains, the mean square is $((3/\sqrt{12.5})^2 + (4/\sqrt{12.5})^2)/2 = 1$. After the gains it is

$$\frac{(6/\sqrt{12.5})^2 + (2/\sqrt{12.5})^2}{2} = \frac{36 + 4}{2(12.5)} = \frac{8}{5} = 1.6.$$

The first gain multiplies its coordinate's squared contribution by four. The second multiplies its contribution by one quarter. The resulting mean square depends on how much each feature contributed before the gains.

3. Change the sign of a feature

In the worked SwiGLU example, the gate input at the first feature was zero, so the feature with value 3 contributed nothing. The exercise replaces the gate input with -1 at that coordinate. The second gate input stays at 2, and the two features are still $\mathbf{u} = (3, -1)$.

For a scalar input a, the **SiLU** activation is $a/(1 + \exp(-a))$. Its denominator is positive, so the activated gate has the sign of a. Apply SiLU separately to each gate coordinate, then multiply by the corresponding entry of $\mathbf{u}$.

Feature	Gate input	Activated gate	Feature value	Product
1	-1	$-1/(1+e) \approx -0.268941$	3	$-3/(1+e) \approx -0.806824$
2	2	$2/(1+e^{-2}) \approx 1.761594$	-1	$-2/(1+e^{-2}) \approx -1.761594$

The first gate turns a positive feature into a negative product. The second gate is positive and greater than one, so its product stays negative and grows in magnitude. Let $a = -3/(1+e)$ and $b = -2/(1+e^{-2})$ denote the two products. Multiply the row vector (a, b) by the chapter's down projection,

$$W_{down} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.$$

The first row gets coefficient a, and the second gets coefficient b. Adding the two scaled rows gives the four output coordinates.

$$(a, b)W_{down} = (a, 0, a, 0) + (0, b, 0, -b) = (a, b, a, -b).$$

The update is approximately $(-0.806824, -1.761594, -0.806824, 1.761594)$. In particular, the fourth output is positive because the matrix multiplies the negative second product by -1. The change to the first gate appears in both the first and third output coordinates because the first row has a one in both columns.

4. Match a parameter budget

Suppose a token has d features and the conventional feed-forward network expands it to f features. The input projection has df weights and the output projection has fd weights. Their total is $2df$. The activation between them adds no learned weights. The exercise excludes biases.

SwiGLU uses two input projections because it computes both a gate and a feature vector. If both have width m, each input projection has dm weights. The down projection has md weights. The total is therefore $3dm$. To match the conventional count, set $3dm = 2df$ and divide by $3d$. This gives the ideal intermediate width $m = 2f/3$.

Model width d	Conventional width f	Conventional count $2df$	Ideal SwiGLU width m
12	48	$2 \cdot 12 \cdot 48 = 1,152$	$2 \cdot 48/3 = 32$
10	40	$2 \cdot 10 \cdot 40 = 800$	$2 \cdot 40/3 = 80/3$

At width $d = 12$, $m = 32$ is already an integer. Each SwiGLU projection has $12 \cdot 32 = 384$ weights, and the three projections have $3 \cdot 384 = 1,152$ weights, an exact match.

At width $d = 10$, the ideal width $80/3 \approx 26.67$ is fractional. The multiples of eight on either side are 24 and 32. Rounding **up** gives 32, so each SwiGLU projection has $10 \cdot 32 = 320$ weights and the total is **960**. This is 160 above the original 800, a 20% increase. The increase comes from the width constraint. Each additional intermediate feature adds d weights to each of the three projections, so at $d = 10$ each width step adds 30 weights.

5. Check the sign in RoPE

The unrotated query pair $\mathbf{q} = (1, 0)^\top$ points along the horizontal axis, and the unrotated key pair $\mathbf{k} = (0, 1)^\top$ points along the vertical axis. The original dot product of those two vectors is zero. Rotary position embeddings turn each pair by an angle proportional to its position before taking the dot product. Here the frequency is $\omega = \pi/6$, or 30° per position. A positive angle turns counterclockwise.

With the chapter's rotation convention,

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Rotating the horizontal unit vector gives the first column of $R(\theta)$. Rotating the vertical unit vector gives the second column. At query position $m = 3$, the angle is $\pi/2$, so the query becomes $(0, 1)^\top$. At key position $n = 1$, the angle is $\pi/6$, so the key becomes $(-1/2, \sqrt{3}/2)^\top$. The dot product is

$$(0, 1) \cdot (-1/2, \sqrt{3}/2) = \sqrt{3}/2.$$

To calculate the score with just one rotation, bring the query rotation to the key side of the dot product. The query rotation is then transposed. A transposed rotation reverses the angle, and successive rotations add their angles. This gives

$$(R(m\omega)\mathbf{q})^\top R(n\omega)\mathbf{k} = \mathbf{q}^\top R(-m\omega)R(n\omega)\mathbf{k} = \mathbf{q}^\top R((n-m)\omega)\mathbf{k}.$$

The relative angle is $(1-3)\pi/6 = -\pi/3$. Multiplying $R(-\pi/3)$ by $\mathbf{k}$ takes the second column, $(\sqrt{3}/2, 1/2)^\top$. The original query picks out its first coordinate, $\sqrt{3}/2$, exactly as in the calculation with two rotations.

Now shift both positions by two, keeping the unrotated content vectors fixed. The query moves to position 5 and turns by $5\pi/6$, so it becomes $(-\sqrt{3}/2, 1/2)^{\top}$. The key moves to position 3 and turns by $\pi/2$, so it becomes $(-1, 0)^{\top}$. Their dot product is $(-\sqrt{3}/2)(-1) + (1/2)(0) = \sqrt{3}/2$. In the relative calculation, the new difference is $3 - 5 = -2$. The angle remains $-\pi/3$ and the score remains $\sqrt{3}/2$.

This choice of content vectors makes the sign test sensitive. The first coordinate of the rotated vertical key is $-\sin\theta$, so reversing the angle changes the sign of the score. Using $(m-n)\omega = +\pi/3$ would give $-\sqrt{3}/2$ both before and after the shift. The equal-vector example, with both content vectors pointing horizontally, instead reads $\cos\theta$ from the rotated key. Cosine is even, so that example returns $1/2$ for either sign. The shared shift checks dependence on the relative position, and the orthogonal vectors check the sign of that position difference.

12. Design a matched comparison (optional)

Suppose we want to predict the next word in *the fox finds the bell near the gate*. After reading *the fox*, the model can learn that *finds* is likely. Later in the sentence, it may know that a place name comes next without knowing which place the generator picked. We will train two causal decoders on the same next-token task and ask whether the choice of feed-forward network affects their held-out loss. This is a plan for an experiment.

Use the chapter's model width $d = 16$, two blocks, and two attention heads. Put a bias-free two-projection feed-forward network with a ReLU between its projections in one decoder. ReLU replaces negative coordinates with zero. Give it an intermediate width of 48. Put a bias-free SwiGLU with intermediate width 32 in the other decoder. Match the number of weights by using the fact that SwiGLU has a separate gate and up projection.

Feed-forward network	Intermediate width	Weights in one block	Weights in two blocks
Two projections with ReLU	48	$2 \cdot 16 \cdot 48 = 1,536$	3,072
Three projections with SwiGLU	32	$3 \cdot 16 \cdot 32 = 1,536$	3,072

The two networks have the same number of learned feed-forward weights and both return a vector of width 16. Keep the rest of the block fixed, including pre-RMSNorm, the residual additions, rotary attention, the causal and padding masks, and the final normalization. Use the same token embeddings and vocabulary projection design, with separate copies of their weights in the two models. The vocabulary of the experiment may be larger than the chapter's 17-token vocabulary, but the two models share it and therefore have the same total parameter count.

For a reproducible corpus, fix lists of 25 subjects, 20 objects, and 30 places. Each subject has one of five verbs assigned by its list index modulo five. A scene is the tuple of subject, object, and place, so there are $25 \cdot 20 \cdot 30 = 15,000$ distinct scenes. Render each scene as *the fox finds the bell near the gate* and as *today near the gate the fox finds the bell*, substituting that scene's words into both templates. The verb and sentence structure give the model predictable patterns, while the object and place can vary independently. Keep the vocabulary lists and generator with the experiment so that another student can reconstruct the text.

Shuffle the scene IDs once with seed 807 and reserve 12,000 scenes for training, 1,500 for validation, and 1,500 for testing. Render both templates from each scene within its assigned split, giving 24,000, 3,000, and 3,000 sentences respectively. Keeping the two renderings together prevents a held-out scene from appearing in training in a different order. Fit one word vocabulary on the training text, add shared BOS, EOS, PAD, and unknown-token IDs, and use the same mapping in every run. Verify that all ordinary words occur in the training split. Prepend BOS and append EOS before shifting. Each input predicts the following token, including EOS at the end, and right padding contributes no labels.

For each of five paired seeds, 101 through 105, create both models from the same initial tensors for every shared component. Draw the feed-forward weights independently with a uniform distribution between $-1/\sqrt{f}$ and $1/\sqrt{f}$ for each linear map, where the fan-in f is the number of input features to that map. Use the same shuffled batch order within each pair and keep dropout at zero. Train both models with float32 tensors on one CPU thread. Use a fresh AdamW optimizer for every run, with learning rate 3×10^{-4}, betas $(0.9, 0.999)$, and weight decay 0.01. Train for exactly 3,000 updates with batches of 32 sentences. When an epoch ends, reshuffle training indices and continue. The two models in a pair see the same batches, so their exposure to valid target tokens matches as well as their update counts.

Measure validation loss every 250 updates for the learning curves and commit in advance to the checkpoint after update 3,000. Once all runs finish, score that checkpoint on the untouched test split. In evaluation mode, let N_b be the number of valid targets in batch b and let S_b be the sum of their negative log probabilities. Compute the test metric as

$$NLL_{\text{token}} = \frac{\sum_b S_b}{\sum_b N_b}.$$

The unit is nats per valid next token because the logarithm is natural. If a batch contains 20 graded positions and another has 40, the second contributes twice as many targets to the final average. Summing token losses and counts before dividing gives each target equal weight. Record both test NLLs for each seed and calculate the paired difference, SwiGLU minus ReLU. Report its mean and the spread across seeds. A negative difference favors SwiGLU on this corpus under the stated training budget.

Measure execution time in a separate benchmark. Use the same device, precision, thread count, software, and sequence shapes for both models. For training, keep a representative batch already on the device, warm up for 25 steps, then time intervals of 100 steps, each including the forward pass, backward pass, and optimizer update. Synchronize an accelerator at the start and end of each timed interval so queued work is included. Run ten intervals per model, alternating which model goes first and report median milliseconds per update and valid targets per second. For inference, time evaluation-mode forward passes with gradients disabled, stating the batch size and prefix length. If the feed-forward branch alone is of interest, also time the two branches on the same input tensor separately from the full decoder. Matching the weights controls the parameter budget in this comparison. Runtime also depends on the activation, the extra gate product, the shapes of the matrix operations, and how the implementation runs those operations.